Agentic Programming: Engineering Reality and IR Construction

From `phi` Review to `-O0/-O3` and `IRBuilder`

Jian Weng

CEMSE, KAUST

Week 11, Session 1

Today's Agenda

1. A quick reading and why the story matters
2. Recap: SSA, `phi`, and merge points
3. A small `sum_array` case under `-O0` and `-O3`
4. What optimization changes in the IR
5. How to build IR with LLVM `IRBuilder`

A Quick Reading

Reading:

- https://substack.com/home/post/p-193230263

Story is quick, but the lesson is bitter.

- an AI-assisted workflow still failed close to a real deadline
- generated LaTeX was not reliable enough
- when the toolchain is brittle, one bad artifact can block everything
- if nobody can debug the raw tool output, the last mile becomes very expensive

The Additional Lesson for This Class

Last time I leaned a bit too hard on "use AI systematically".

That still helps, but we should add one correction:

- AI does not remove the need for engineering taste
- 95% success can hide the most dangerous 5%
- the remaining 5% may take longer than the first 95%
- compiler work is especially unforgiving because each stage depends on the previous artifact

Recap from `w10d2`

Where were we last time?

- `SSA` gives us define-once value versions
- `phi` merges different incoming definitions at a CFG convergence
- dominance explains where a definition is guaranteed to be available
- dominance frontier explains where different definitions may need merging

Today we continue from there and move toward actual IR construction.

A Small `phi` Reminder

```
        B0
      /    \
     v      v
    B1      B2
     \      /
      v    v
        B3
```

```
B1: y1 = 1
B2: y2 = 2
B3: y3 = phi([y1, B1], [y2, B2])
```

- `phi` is selected by the incoming edge
- it is the bridge between CFG structure and SSA values

In Our Course, The Case Is Often Simpler

The textbook algorithm is general.

Your generated IR is usually more structured.

- blocks often have at most two predecessors
- a block with two predecessors is usually a merge point
- if both incoming paths can define the same logical variable, inspect that merge block first
- that is often enough intuition before implementing the full general algorithm

This is not the full theorem.

It is the practical shortcut for our compiler case study.

Case Study: Accumulating an Array

Source:

```c
int sum_array(int *a, int n) {
  int sum = 0;
  for (int i = 0; i < n; ++i) {
    sum += a[i];
  }
  return sum;
}
```

Useful inspection commands:

```
clang -S -emit-llvm -O0 -Xclang -disable-O0-optnone sum.c -o sum-O0.ll
clang -S -emit-llvm -O3 sum.c -o sum-O3.ll
```

What `-O0` Looks Like

```
%5 = alloca i32, align 4
%6 = alloca i32, align 4
store i32 0, ptr %5, align 4
store i32 0, ptr %6, align 4
br label %7

7:
  %8 = load i32, ptr %6, align 4
  %10 = icmp slt i32 %8, %9
  br i1 %10, label %11, label %22

11:
  %17 = load i32, ptr %5, align 4
  %18 = add nsw i32 %17, %16
  store i32 %18, ptr %5, align 4
```

- `sum` and `i` both live in stack slots
- every loop iteration loads and stores them again
- the IR stays very close to the source-level mutation model

What `-O3` Looks Like

```
7:
  %8 = phi i64 [ 0, %4 ], [ %13, %40 ], [ %46, %56 ]
  %9 = phi i32 [ 0, %4 ], [ %38, %40 ], [ %57, %56 ]
  br label %61

61:
  %62 = phi i64 [ %67, %61 ], [ %8, %7 ]
  %63 = phi i32 [ %66, %61 ], [ %9, %7 ]
  %65 = load i32, ptr %64, align 4
  %66 = add nsw i32 %65, %63
  %67 = add nuw nsw i64 %62, 1
```

- loop-carried state becomes SSA values
- `phi` carries the evolving `i` and `sum`
- this is much easier for later optimization passes to analyze

–O3 Does More Than Remove Loads and Stores

In the same function, LLVM also recognizes a reduction and vectorizes it:

```
%16 = phi <4 x i32> [ zeroinitializer, %12 ], [ %28, %14 ]
%17 = phi <4 x i32> [ zeroinitializer, %12 ], [ %29, %14 ]
...
%28 = add <4 x i32> %24, %16
%29 = add <4 x i32> %25, %17
...
%38 = tail call i32 @llvm.vector.reduce.add.v4i32(<4 x i32> %37)
```

- the CFG is reorganized into vector loop plus cleanup loop
- once values are explicit in SSA, loop optimizations become much more direct

What Changed Between `-O0` and `-O3`?

- stack-style locals became SSA values
- repeated `load` / `store` traffic disappeared
- loop state is represented by `phi` instead of mutable slots
- the optimizer can now reason about def-use chains directly
- after that, transformations such as vectorization become possible

This is why SSA is not only elegant.

It is operationally useful.

Build Up Your IR

`IRBuilder` is LLVM's construction helper.

It is not a different IR.

Think of it as:

- a cursor for where new IR is emitted
- a convenience layer for creating instructions
- a practical API for building functions, blocks, branches, and `phi`

If you want your frontend to emit LLVM IR, this is the API you will touch every day.

What Is an Insert Point?

The insert point is where the next instruction will be inserted.

- usually at the end of a basic block
- can also be before a specific instruction
- controlled by `builder.SetInsertPoint(...)`

```
auto *entry = llvm::BasicBlock::Create(ctx, "entry", fn);
builder.SetInsertPoint(entry);
auto *sum = builder.CreateAdd(lhs, rhs, "sum");
builder.CreateRet(sum);
```

Without the correct insert point, your IR ends up in the wrong block.

Creating the Outer Shell

```
auto *i32 = builder.getInt32Ty();
auto *fty = llvm::FunctionType::get(i32, {i32, i32}, false);
auto *fn = llvm::Function::Create(
    fty,
    llvm::Function::ExternalLinkage,
    "add",
    module);

auto *entry = llvm::BasicBlock::Create(ctx, "entry", fn);
builder.SetInsertPoint(entry);
```

- create the function type first
- then create the function
- then create an entry block
- then point the builder at that block

Creating Instructions

Without `IRBuilder`:

```
llvm::BinaryOperator::Create(
    llvm::Instruction::Add,
    lhs,
    rhs,
    "sum",
    insertPoint);
```

With `IRBuilder`:

```
builder.SetInsertPoint(insertPoint);
auto *sum = builder.CreateAdd(lhs, rhs, "sum");
```

- the builder version is shorter and easier to compose
- the raw constructor version still exists for lower-level control

Basic Blocks First, Then Builder

In practice, basic blocks are usually created with:

```
auto *thenBB = llvm::BasicBlock::Create(ctx, "then", fn);
auto *elseBB = llvm::BasicBlock::Create(ctx, "else");
auto *mergeBB = llvm::BasicBlock::Create(ctx, "merge");
```

Then the builder is moved into them:

```
builder.SetInsertPoint(thenBB);
...
builder.SetInsertPoint(elseBB);
...
builder.SetInsertPoint(mergeBB);
```

So the workflow is:

```
create block -> set insert point -> emit instructions
```

A Typical `if / else` Recipe

```
builder.CreateCondBr(cond, thenBB, elseBB);

builder.SetInsertPoint(thenBB);
Value *y1 = builder.getInt32(1);
builder.CreateBr(mergeBB);

fn->insert(fn->end(), elseBB);
builder.SetInsertPoint(elseBB);
Value *y2 = builder.getInt32(2);
builder.CreateBr(mergeBB);

fn->insert(fn->end(), mergeBB);
builder.SetInsertPoint(mergeBB);
auto *y = builder.CreatePHI(builder.getInt32Ty(), 2, "y");
y->addIncoming(y1, thenBB);
y->addIncoming(y2, elseBB);
```

This is the practical pattern that connects today's topic back to last time's `phi`.

What This Means for Your Project

Your IR generator should gradually learn to do all of this:

- keep one `LLVMContext`, one `Module`, and one `IRBuilder`
- let each expression return an `llvm::Value *`
- create basic blocks explicitly for control flow
- move the insert point carefully as control changes
- create `phi` whenever one result can come from multiple predecessors

When something breaks, print the IR and inspect it directly.

Takeaways

- AI can help you generate code, but it cannot replace engineering judgment
- `-O0` often exposes stack-style mutation, while `-O3` recovers SSA loop state
- `phi` is not just theory; it is how optimized loops carry values forward
- `IRBuilder` is the practical API for emitting LLVM IR in your compiler
- once you can control blocks and insert points, the rest becomes much easier